Ejercicio7

Las pruebas están en Ejercicio7Test.

PARTE 1

1.1 Dibujo del grafo
  Está en el jpeg "Ejercicios 1.1 y 2.1.jpeg"

1.2 DIJKSTRA
  En el jpeg "Ejercicios 1.2 y 1.3.jpeg"

1.3 Predecesores y Caminos
  En el jpeg "Ejercicios 1.2 y 1.3.jpeg"

1.4 Imprimir camino de predecesores

Lenguaje natural:
Se parte del destino y se va retrocediendo con el vector de predecesores hasta
llegar al origen, guardando cada vértice al principio de una lista. Al terminar,
la lista queda en orden origen -> destino. Si en algún momento no hay predecesor
(se llegó a nulo sin pasar por el origen), el destino no es alcanzable.

Precondiciones: el vector de predecesores salió de ejecutar Dijkstra con ese origen.

Postcondiciones:
- No cambia el vector.
- Devuelve la lista de vértices del camino origen -> destino, o vacía si no hay camino.

Pseudocódigo:

```
camino(predecesor, origen, destino) devuelve Lista
Comienzo
    camino <- lista vacía
    actual <- destino
    MIENTRAS actual <> nulo AND actual <> origen HACER
        camino.agregarAlPrincipio(actual)
        actual <- predecesor[actual]
    FIN MIENTRAS
    SI actual = nulo ENTONCES
        devolver lista vacía              // no se llegó al origen
    FIN SI
    camino.agregarAlPrincipio(origen)
    devolver camino
Fin
```

Orden del tiempo de ejecución: O(V), el camino tiene como máximo V vértices.

Implementado en Ejercicio7.camino(...) (para un Map de predecesores) y en
DijkstraResult.getPath(...).

==========

PARTE 2

2.1 Lista de adyacencias
  En el jpeg "Ejercicios 1.1 y 2.1.jpeg"

2.2 Floyd

Lenguaje natural:
Se usa una matriz de costos D (inicialmente el peso de cada arista, 0 en la diagonal e infinito donde no hay arista) y una matriz de predecesores P, donde P[i][j] es el vértice que viene justo antes de j en el mejor camino conocido de i a j. Para cada vértice k, se prueba si ir de i a j pasando por k es más barato que lo que ya se tenía. Si lo es, se actualiza D[i][j] y P[i][j] pasa a ser P[k][j] (el vértice anterior a j en el tramo k -> j). Al terminar con todos los k, D tiene los costos mínimos de todos los pares.

Precondiciones: no hay ciclos de costo negativo.

Postcondiciones:
- El grafo no cambia.
- D[i][j] es el costo mínimo de i a j (infinito si no hay camino).
- P permite reconstruir cada camino mínimo.

Pseudocódigo:

```
floyd(grafo) devuelve (D, P)
Comienzo
    PARA i DESDE 1 HASTA n HACER
        PARA j DESDE 1 HASTA n HACER
            SI i = j ENTONCES D[i][j] <- 0 SINO D[i][j] <- infinito FIN SI
            P[i][j] <- nulo
        FIN PARA
    FIN PARA
    PARA CADA arco (i -> j) con peso w EN aristas(grafo) HACER
        D[i][j] <- w
        P[i][j] <- i
    FIN PARA

    PARA k DESDE 1 HASTA n HACER
        PARA i DESDE 1 HASTA n HACER
            PARA j DESDE 1 HASTA n HACER
                SI D[i][k] + D[k][j] < D[i][j] ENTONCES
                    D[i][j] <- D[i][k] + D[k][j]
                    P[i][j] <- P[k][j]
                FIN SI
            FIN PARA
        FIN PARA
    FIN PARA
    devolver (D, P)
Fin
```

Orden del tiempo de ejecución: tres bucles anidados de n = V vueltas, O(V^3).
Espacio: O(V^2) por las dos matrices.

REPRESENTACION EN ARCHIVOS jpeg: "Ejercicio 2.2 parte 1.jpeg" y "Ejercicio 2.2 parte 2.jpeg"

2.3 Recuperar el camino con el vector de predecesores
Lenguaje natural:
Para ir de i a j se mira P[i][j]: ese es el vértice anterior a j. Después se mira P[i][anterior], y así hasta llegar a i. Los vértices encontrados, leídos al revés, son el camino.

Pseudocódigo:

```
caminoFloyd(P, i, j) devuelve Lista
Comienzo
    SI P[i][j] = nulo AND i <> j ENTONCES
        devolver lista vacía              // no hay camino
    FIN SI
    camino <- lista vacía
    actual <- j
    MIENTRAS actual <> i HACER
        camino.agregarAlPrincipio(actual)
        actual <- P[i][actual]
    FIN MIENTRAS
    camino.agregarAlPrincipio(i)
    devolver camino
Fin
```

2.4 Camino de menor costo desde A a E

- Inicialmente D[A][E] = infinito

- En k = C se prueba A -> C -> E: D[A][C] + D[C][E] = 1 + 1 = 2 < infinito.
  Se actualiza D[A][E] = 2 y P[A][E] = P[C][E] = C.

- En k = D y k = E no mejora.

Con la matriz P: P[A][E] = C, P[A][C] = A (origen).

Camino de menor costo: A -> C -> E, costo 2.

2.5 Excentricidad

  Columna A: 6 (B), 4 (C), 9 (D), 3 (E)   -> excentricidad(A) = 9
  Columna B: 3 (A), 2 (C), 7 (D), 6 (E)   -> excentricidad(B) = 7
  Columna C: 1 (A), 7 (B), 5 (D), 4 (E)   -> excentricidad(C) = 7
  Columna D: 4 (A), 10 (B), 8 (C), 7 (E) -> excentricidad(D) = 10
  Columna E: 2 (A), 3 (B), 1 (C), 6 (D)   -> excentricidad(E) = 6

2.6 Centro del grafo

El centro es el vértice de menor excentricidad. La menor es 6, la de E.
El centro del grafo es el vértice E.

Pseudocódigo de las dos operaciones (después de ejecutar floyd):

```
excentricidad(D, v) devuelve numero
Comienzo
    max <- 0
    PARA CADA otro EN vertices(grafo) HACER
        SI D[otro][v] > max ENTONCES max <- D[otro][v] FIN SI
    FIN PARA
    devolver max
Fin

centro(grafo) devuelve vértice
Comienzo
    D <- floyd(grafo)
    centro <- nulo
    menor <- infinito
```

```
    PARA CADA v EN vertices(grafo) HACER
        exc <- excentricidad(D, v)
        SI centro = nulo OR exc < menor ENTONCES
            centro <- v
            menor <- exc
        FIN SI
    FIN PARA
    devolver centro
Fin
```

Orden: floyd es O(V^3) y calcular las V excentricidades es O(V^2). Total O(V^3).
Precondición de centro: el grafo no es vacío. Si hay empate, se devuelve el primer vértice en orden de inserción.